BARUCH COLLEGE

Department of Mathematics • Master of Financial Engineering

DEEP LEARNING IN FINANCE | Lecture Series

Deep Learning Statistical Arbitrage

Part IV

Oualid Missaoui

Adjunct Faculty, Baruch College

Senior Quantitative Researcher, Caerus Commodities Trading — New York

Roadmap

The Big Picture

Portfolios, Signals, and Allocation

The CNN + Transformer Architecture

What the Attention Learns

Empirical Results

The paper



Jorge Guijarro-Ordonez
LinkedIn



Markus Pelger
Stanford



Greg Zanotti
LinkedIn

Deep Learning Statistical Arbitrage*

Jorge Guijarro-Ordonez† Markus Pelger‡ Greg Zanotti§

This draft: September 25, 2022
First draft: March 15, 2019

Abstract

Statistical arbitrage exploits temporal price differences between similar assets. We develop a unifying conceptual framework for statistical arbitrage and a novel data driven solution. First, we construct arbitrage portfolios of similar assets as residual portfolios from conditional latent asset pricing factors. Second, we extract their time series signals with a powerful machine-learning time-series solution, a convolutional transformer. Lastly, we use these signals to form an optimal trading policy, that maximizes risk-adjusted returns under constraints. Our comprehensive empirical study on daily US equities shows a high compensation for arbitrageurs to enforce the law of one price. Our arbitrage strategies obtain consistently high out-of-sample mean returns and Sharpe ratios, and substantially outperform all benchmark approaches.

Keywords: statistical arbitrage, pairs trading, machine learning, deep learning, big data, stock returns, convolutional neural network, transformer, attention, factor model, market efficiency, investment.
JEL classification: C14, C38, C55, G12

arXiv:2106.04028

Roadmap

The Big Picture

Portfolios, Signals, and Allocation

The CNN + Transformer Architecture

What the Attention Learns

Empirical Results

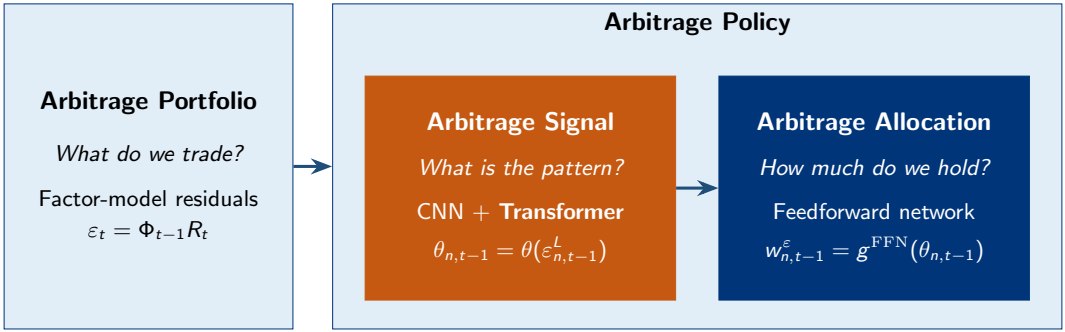
Overview

- Take any big equity universe.
- Strip out all risk premia with an asset-pricing model, so you are left with residual "mispricing" portfolios.
- Then train a deep time-series model (**CNN + Transformer**) *not to predict returns*, but directly to **maximize the Sharpe ratio** (or mean–variance) of a long–short portfolio of those residuals, under realistic constraints.

Today's focus: the **Transformer-based architecture**. How does the attention machinery from Part I become a trading signal? The classical benchmarks are mentioned only as yardsticks.

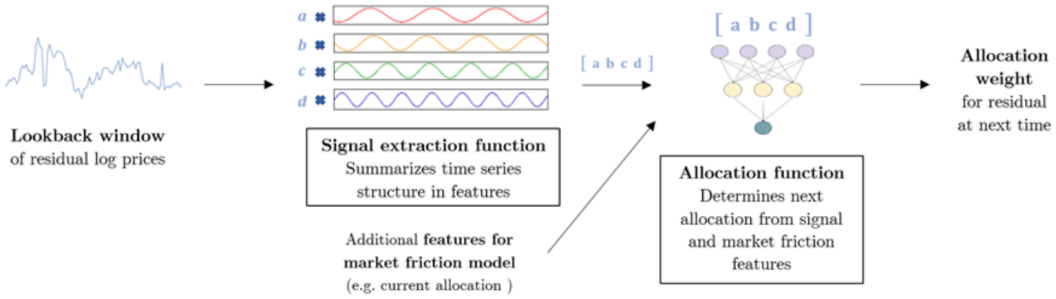
▶ **In finance:** the same encoder that translates sentences here reads the last 30 days of a residual, and it is graded on portfolio Sharpe rather than on forecast error.

Architecture: three decisions



▶ **In finance:** this split separates *what to trade* from *when and how much*, so each piece can be swapped and compared on its own.

Conceptual arbitrage model



The model takes the last L cumulative returns of a residual portfolio and outputs the allocation weight for that residual at the next time. It is a composition of a **signal extraction function** and an **allocation function**.

Source: Guijarro-Ordonez, Pelger, Zanotti (2022), Figure 1.

Roadmap

The Big Picture

Portfolios, Signals, and Allocation

The CNN + Transformer Architecture

What the Attention Learns

Empirical Results

Arbitrage portfolios via residuals

Factor model and factor returns

- Excess returns of N assets: $R_t \in \mathbb{R}^N$. A K-factor model:

$$R_t = \beta_{t-1} F_t + \varepsilon_t$$

  - $\beta_{t-1} \in \mathbb{R}^{N \times K}$: loadings, known at $t-1$
  - $F_t \in \mathbb{R}^K$: factor returns at t
  - $\varepsilon_t \in \mathbb{R}^N$: residual returns

- Given β_{t-1} and R_t, estimate F_t by **OLS**:

$$\hat{F}_t = \arg\min_f \|R_t - \beta_{t-1} f\|_2^2 = (\beta_{t-1}^\top \beta_{t-1})^{-1} \beta_{t-1}^\top R_t$$

Residuals as a projection

- **Projection onto factor space**:

$$P_{\beta,t-1} = \beta_{t-1}(\beta_{t-1}^\top \beta_{t-1})^{-1} \beta_{t-1}^\top$$

- Plug in $\hat{F}_t$ and factor out R_t:

$$\varepsilon_t = R_t - \beta_{t-1}\hat{F}_t = (I_N - P_{\beta,t-1}) R_t$$

- So we define:

$$\Phi_{t-1} := I_N - \beta_{t-1}(\beta_{t-1}^\top \beta_{t-1})^{-1} \beta_{t-1}^\top$$

Residuals are factor-neutral arbitrage portfolios

Factor neutrality

Since $\beta_{t-1}^\top \beta_{t-1} \in \mathbb{R}^{K\times K}$ is invertible,

$$\begin{aligned}\beta_{t-1}^\top \varepsilon_t &= \beta_{t-1}^\top \Phi_{t-1} R_t \\ &= \beta_{t-1}^\top R_t - \underbrace{\beta_{t-1}^\top \beta_{t-1} (\beta_{t-1}^\top \beta_{t-1})^{-1}}_{I_K} \beta_{t-1}^\top R_t \\ &= 0.\end{aligned}$$

Conclusion: residual portfolios are orthogonal to the factors, i.e. they have **zero factor exposure**.

Each residual is a portfolio

- With basis vector $e_n \in \mathbb{R}^N$, define the **arbitrage-portfolio weights**

$$w^{(n)} := \Phi_{t-1}^\top e_n \in \mathbb{R}^N.$$

- Its return is the residual itself:

$$w^{(n)\top} R_t = e_n^\top \Phi_{t-1} R_t = \varepsilon_{n,t}.$$

- $\varepsilon_{n,t}$ is the return of a **self-financing, factor-neutral long–short portfolio**.
- Each "asset" in residual space is an arbitrage portfolio over the original N stocks.

Factor models used to construct arbitrage portfolios

Family	K	Factors	Estimation
Fama–French (pre-specified)	1, 3, 5, 8	Market; size, value; profitability, investment; momentum, short- and long-term reversal	Daily rolling regression of each stock on the factors, previous 60 days
PCA (latent, unconditional)	1, 3, 5, 8, 10, 15	First K principal components of the correlation matrix	PCs on the last 252 days; loadings by regression on the last 60 days
IPCA (latent, conditional)	1, 3, 5, 8, 10, 15	Loadings linear in 46 firm characteristics: $\beta_{i,t} = \Gamma^\top z_{i,t}$	Re-estimated yearly on the last 240 months
Zero-factor	0	None: raw excess stock returns	Control case

All estimates use information up to $t-1$ only, so residuals are obtained **out of sample**.

▶ **In finance:** any factor model plugs in here. The Transformer never sees the factors, only the residual path they leave behind.

Arbitrage signal

Goal: turn the **recent history of each arbitrage portfolio** into a compact **trading signal**.

What goes in and what comes out

- For portfolio n on day t, collect the last L daily residual returns, its recent PnL history:

$$\varepsilon^{L}_{n,t-1} = (\varepsilon_{n,t-L}, \ldots, \varepsilon_{n,t-1})$$

- Apply a **time-series model**:

$$\theta_{n,t-1} = \theta(\varepsilon^{L}_{n,t-1})$$

- Simple: a mean-reversion fit gives a z-score.
- Advanced: **CNN + Transformer** gives a learned vector capturing trend and reversal patterns.

Practical assumptions

- **Short-horizon stationarity**: patterns in the last L days inform the very near future.
- **Finite memory**: the first L lags are enough, and $\theta_{n,t-1}$ compresses all we need.

Use in a trading system

1. Each day, for each n: pull L residuals, compute $\theta_{n,t-1}$.
2. Feed $\theta_{n,t-1}$ to the **allocation network**.
3. Convert to stock weights via $\Phi^{\top}$.

▶ **In finance:** benchmark lookback is $L = 30$ days; the paper reports very comparable results with $L = 60$.

Arbitrage trading = signal + allocation + tradable weights

Allocation function

$$w^{\varepsilon} : \theta_{n,t-1} \mapsto w^{\varepsilon}_{n,t-1}$$

Tradable stock weights

$$w^{R}_{t-1} = \frac{(w^{\varepsilon}_{t-1})^{\top}\Phi_{t-1}}{\left\|(w^{\varepsilon}_{t-1})^{\top}\Phi_{t-1}\right\|_1}$$

The $\|\cdot\|_1$ normalization enforces a **leverage constraint**: absolute weights sum to one.

Joint optimization of signal and allocation

$$\max_{w^{\varepsilon}\in\mathcal{W},\,\theta\in\Theta} \mathbb{E}_{t-1}\Big[U\big((w^{R}_{t-1})^{\top}R_t\big)\Big] \quad \text{s.t.} \quad w^{\varepsilon}_{t-1} = w^{\varepsilon}\big(\theta(\varepsilon^{L}_{t-1})\big)$$

Objectives used in the paper

$$\max \frac{\mathbb{E}\big[(w^{R}_{t-1})^{\top}R_t\big]}{\sqrt{\mathrm{Var}\big((w^{R}_{t-1})^{\top}R_t\big)}} \quad \text{(Sharpe), or}$$

$$\max \mathbb{E}\big[(w^{R}_{t-1})^{\top}R_t\big] - \gamma\,\mathrm{Var}\big((w^{R}_{t-1})^{\top}R_t\big) \quad \text{(mean–variance)}$$

Key takeaway: the split into θ and w^{ε} is *not uniquely identified*, since many pairs give the same policy. Empirically, **most performance comes from a rich time-series signal** θ; once a good signal exists, many allocation rules can exploit it.

From arbitrage-portfolio weights to stock weights

Why Φ shows up again

The model works in arbitrage space: $w^{\varepsilon}_{t-1} \in \mathbb{R}^N$ says how much of each arbitrage portfolio we hold. The PnL then rewrites as a position in stocks:

$$\begin{aligned}\mathrm{PnL}_t &= (w^{\varepsilon}_{t-1})^\top \varepsilon_t = (w^{\varepsilon}_{t-1})^\top \Phi_{t-1} R_t \\ &= \left(\Phi^\top_{t-1} w^{\varepsilon}_{t-1}\right)^\top R_t = (w^R_{t-1})^\top R_t\end{aligned}$$

$$w^R_{t-1} := \Phi^\top_{t-1} w^{\varepsilon}_{t-1}$$

- Φ acts on **returns** to create residuals.
- $\Phi^\top$ acts on **weights** to map them to stocks.
- No inverse and no double-counting, only portfolio composition.

ETF analogy

Each residual is a **synthetic ETF** on the stocks A, B:

- $\mathrm{ETF}_1 = 0.8A - 0.4B$, $\mathrm{ETF}_2 = -0.4A + 0.2B$
- Holdings matrix, with one ETF per row:

$$H = \begin{bmatrix} 0.8 & -0.4 \\ -0.4 & 0.2 \end{bmatrix}$$

- ETF returns: $\varepsilon = HR$
- Holding ETF weights w^{ε} means stock weights $w^R = H^\top w^{\varepsilon}$

Φ plays the role of H. This H is in fact Φ for one factor with $\beta = (1, 2)^\top$.

Roadmap

The Big Picture

Portfolios, Signals, and Allocation

The CNN + Transformer Architecture

What the Attention Learns

Empirical Results

Where the Transformer sits among the models

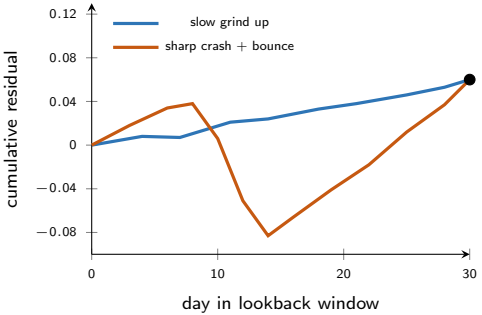
Model	Signal θ	Allocation	Sharpe-trained?	OOS SR
OU + Threshold	Parametric mean-reversion fit	Hard rule	No, evaluation only	0.97
OU + FFN	Same OU parameters	FFN	Allocation only	0.66
Raw path + FFN	None	FFN	End-to-end	1.55
Fourier + FFN	Fixed FFT filter	FFN	Allocation only	1.90
CNN + Transformer + FFN	**Learned local and global patterns**	**FFN**	**End-to-end**	**4.16**

Out-of-sample annualized Sharpe, IPCA residuals with $K = 5$, Jan 2002–Dec 2016 (Tables I and A.IX).

- All the FFN variants share a flexible allocator, so they differ in the **signal**.
- The gap from 1.90 to 4.16 therefore measures what a **learned time-series model** adds over a fixed filter.
- A flexible allocator **cannot compensate** for an uninformative signal.

▶ **In finance:** at $K = 5$ on IPCA, Fourier earns 7.7% at 4.1% vol; CNN + Transformer earns 8.7% at 2.1% vol.

Why a Transformer? The path matters, not just the endpoint



Illustrative paths. Same endpoint, very different histories.

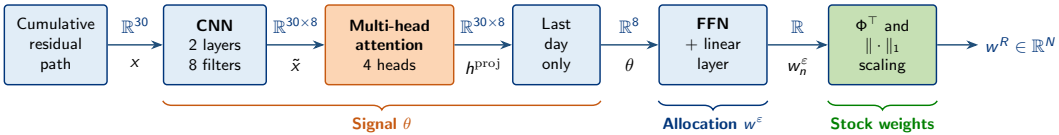
Break the problem in two:

1. **CNN: local filters.** Slides small filters over the path and learns "local up move", "U-shape", "mini-reversal".
2. **Transformer: global dependencies.** Relates those local patterns across the whole window, e.g. "early trend then reversal".
3. **FFN: allocation.** Maps the final representation to a weight.

Parametric and fixed filters prescribe which patterns matter. Here, **all of them are estimated from the data**.

▶ **In finance:** a Fourier filter projects on fixed sinusoids. Attention plays a similar role with *learned* global patterns.

The full pipeline, with benchmark tensor shapes



- **One network, shared by all residuals.** It runs on each path n separately, so assets are a batch dimension.
- **Attention runs across time, not across assets.** The tokens are the L days of one residual.
- Assets interact only at the end, through $\Phi^{\top}$, the $\|\cdot\|_1$ normalization, and the portfolio objective.

▶ **In finance:** sharing weights across stocks handles an unbalanced panel, as names can enter or leave the universe without changing the model.

CNN: learning local residual patterns

Input: cumulative residuals of one arbitrage portfolio, $x = (X_1, \dots, X_L) \in \mathbb{R}^L$ with $X_l = \sum_{j=1}^{l} \varepsilon_{n,t-L-1+j}$.

Layer 1: filter size $D_{\text{size}} = 2$, D filters

$$y^{(0)}_{l,d} = \sum_{m=1}^{D_{\text{size}}} W^{(0)}_{d,m}\, x_{l-m+1}, \qquad x^{(1)}_{l,d} = \mathrm{ReLU}\big(y^{(0)}_{l,d}\big)$$

Layer 2: same idea, applied to $x^{(1)}$

$$y^{(1)}_{l,d} = \sum_{m=1}^{D_{\text{size}}} \sum_{j=1}^{D} W^{(1)}_{d,j,m}\, x^{(1)}_{l-m+1,j}, \qquad x^{(2)}_{l,d} = \mathrm{ReLU}\big(y^{(1)}_{l,d}\big)$$

Output: $\tilde{x} \in \mathbb{R}^{L \times D}$, where column d is the **activation of local pattern** d at each time step.

Implementation details (Appendix B)

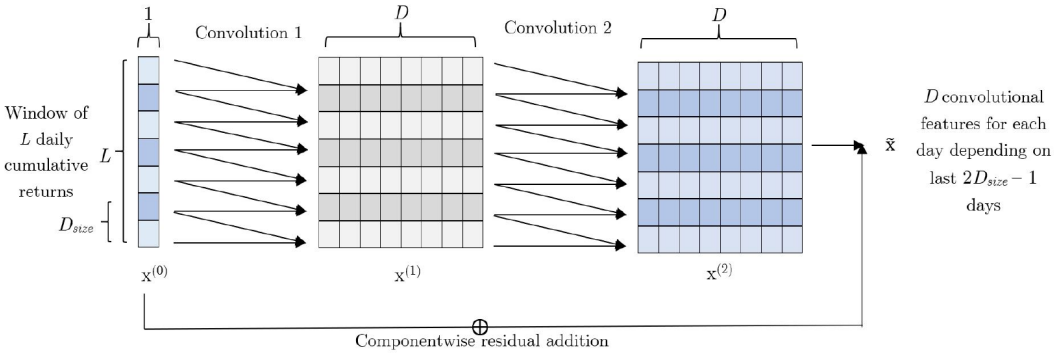
- Bias terms in each layer
- **Instance normalization** before each ReLU
- **Residual connection**: $\tilde{x}_{l,d} = x^{(2)}_{l,d} + x^{(0)}_{l}$

Reading the output

- Each row is a **day**, described by its 8 pattern exposures.
- Each day sees the last $2D_{\text{size}} - 1 = 3$ days.

Link to Part I: the CNN is this model's **embedding layer**. It replaces the word-embedding lookup and turns each day into a token of dimension D.

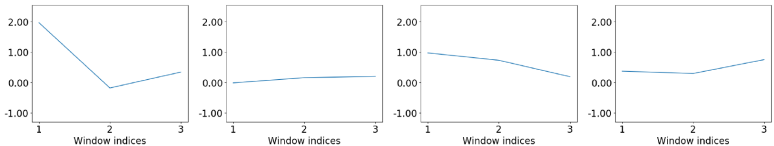
Convolutional network architecture



The input is a window of L daily cumulative returns of a residual. The output is D features for each day, each a nonlinear function of a short block of days that captures a common pattern.

Source: Guijarro-Ordonez, Pelger, Zanotti (2022), Figure 3.

The vocabulary the CNN learns: 8 local patterns



(a) Basic pattern 1 (b) Basic pattern 2 (c) Basic pattern 3 (d) Basic pattern 4

(e) Basic pattern 5 (f) Basic pattern 6 (g) Basic pattern 7 (h) Basic pattern 8

- Patterns 4, 6: **local up-trends**. Patterns 3, 7: **local down-trends**. Patterns 1, 5, 8: **reversals**.
- Enough to build any smooth trend or mean-reversion shape, but no very spiked, sharp changes.

Source: paper, Figure 14. Projections of the nonlinear filters onto linear ones; IPCA 5-factor model, fit once on the first 8 years.

From local to global: attention as pattern exposure

The CNN output is only a local view. The Transformer models **temporal dependencies between local patterns**.

Simplified model. Assume H global patterns. For each $i = 1, \ldots, H$:

$$h_i^{\text{simple}} = \sum_{j=1}^{L} \alpha_{i,j}\, \tilde{x}_j, \qquad \alpha_{i,j} = \alpha_i(\tilde{x}_L, \tilde{x}_j) \in [0,1]$$

- $\alpha_{i,j}$ says **how much the local pattern on day j matters**, given the most recent local pattern $\tilde{x}_L$.
- h_i^{simple} is the **exposure** of this residual to global pattern i.
- A global up-trend puts weight on successive local up-trends; mean reversion on alternating curved patterns.

The analogy to factor models: attention heads are "**pattern factors**" over time, and the signal is the loading of a residual on them, just as β is a loading on cross-sectional factors.

▶ **In finance:** unlike a fixed filter, these weights depend on the path, so the model looks at different days for different residuals.

The actual mechanism: multi-head scaled dot-product attention

1. Project $\tilde{x}$ onto each head $(i = 1, \ldots, H)$

$$V_i = \tilde{x} W_i^V, \quad K_i = \tilde{x} W_i^K, \quad Q_i = \tilde{x} W_i^Q \quad \in \mathbb{R}^{L \times D/H}$$

2. Attend across the L days

$$h_{i,l} = \sum_{j=1}^{L} \alpha_{i,l,j} V_{i,j}, \qquad \alpha_{i,l,j} = \frac{\exp(K_{i,l} \cdot Q_{i,j})}{\sum_{m=1}^{L} \exp(K_{i,l} \cdot Q_{i,m})}$$

3. Concatenate and mix the heads

$$h^{\text{proj}} = \text{Concat}(h_1, \ldots, h_H)\, W^O \in \mathbb{R}^{L \times D}$$

Bias terms omitted. The paper's $K_l \cdot Q_j$ swaps the labels of Part I's $Q_l \cdot K_j$; both matrices are learned, so nothing changes. PyTorch also divides the scores by $\sqrt{D/H}$, as in Vaswani et al.

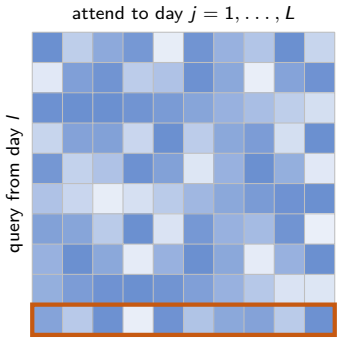
Benchmark dimensions

$\tilde{x}$	30×8
W_i^V, W_i^K, W_i^Q	8×2
V_i, K_i, Q_i	30×2
α_i (one per head)	30×30
h_i	30×2
Concat	30×8
W^O	8×8
h^{proj}	30×8

Same equations as Part I. Only the **tokens** are new: they are days, not words.

▶ **In finance:** each head sees only $D/H = 2$ dimensions, which keeps the parameter count low on a noisy series.

Only the last row becomes the signal



Illustrative α_i for one head

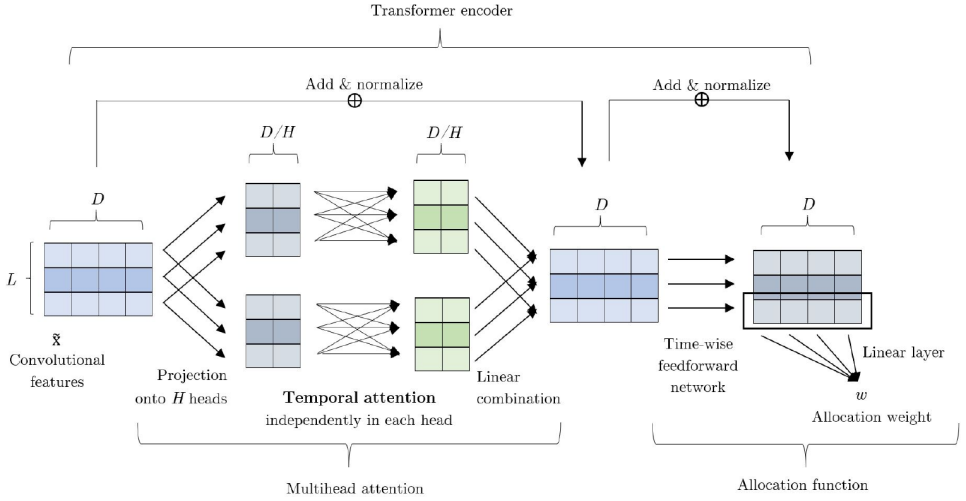
- Each head computes a full $L \times L$ matrix α_i, but the **trading signal is only the last row of** h^{proj}:

$$\theta_{t-1}^{\text{CNN+Trans}} = h_L^{\text{proj}} \in \mathbb{R}^D$$

- The $H \times L$ weights $\alpha_{i,L,j}$ hold **all the global-pattern information** used for trading, so we can **plot and interpret** them.
- Using the full matrix gives similar results, and variable importance selects only h_L.
- **No look-ahead mask is needed**: the whole window precedes the trade.

The main text writes $\mathbb{R}^H$ for this signal; the dimensions in Appendix B give $\mathbb{R}^D$, used here.

Transformer network architecture



Source: Guijarro-Ordonez, Pelger, Zanotti (2022), Figure 4.

From attention to allocation: the rest of the encoder

Encoder block (one layer, following Vaswani et al.)

1. **Add & normalize** around multi-head attention.
2. **Time-wise FFN**: two layers with $2D = 16$ hidden units and the same weights on every day, then add & normalize.
3. **Dropout** at rate 0.25.

Allocation

$$w^{\varepsilon}_{n,t-1} = g^{\mathrm{FFN}}\left(\theta^{\mathrm{CNN+Trans}}_{n,t-1}\right) \in \mathbb{R}$$

Time-wise FFN plus a final linear layer from D to 1.

Why signal and allocation sit inside one encoder

- Time-wise layers act row by row, so row L of the output depends only on row L of h^{proj}.
- Taking the last row **after** the encoder is thus the same as feeding h^{proj}_{L} to the FFN.
- The cut between signal and allocation is a **convention**, chosen for comparison with classical models.

Extra input under trading costs

- The previous allocation is added to the allocation function's inputs.

▶ **In finance:** the paper's Figure 4 labels the time-wise FFN and linear layer as the allocation function, so the arbitrage "signal" is what multi-head attention produces.

What changes relative to the Part I Transformer

	Vaswani et al. (2017)	CNN + Transformer (benchmark)
Task	Translate a sentence	Map one window to one position
Token	Word	Day of a residual path
Embedding	Learned lookup table	2-layer causal CNN, $D_{\text{size}} = 2$
Positional encoding	Sinusoidal	**None**
Structure	Encoder + decoder	**Encoder only**
Layers	6	**1**
d_{model} / heads / FFN width	512 / 8 / 2048	**8 / 4 / 16**
Mask	Look-ahead mask in decoder	None: the whole window is in the past
Output	Softmax over vocabulary	Last row $\rightarrow$ linear $\rightarrow$ real-valued weight
Loss	Cross-entropy, per token	**Portfolio Sharpe**, across all stocks and days

Discussion: with no positional encoding, how can attention favor recent or early days (next section)? Our reading: position reaches attention only implicitly, through the CNN's causal features and the level of the cumulative path, which starts near zero in every window.

No positional encoding appears in the paper's equations or in the replication code's `cnn_transformer.py`.

The benchmark model is tiny

Selected hyperparameters (Table A.II)

Conv. filters D	8
Filter size D_{size}	2
Attention heads H	4
FFN hidden units	$2D = 16$
Dropout	0.25
Lookback L	30 days
Rolling training window	1000 days
Retraining frequency	125 days
Batch size	125 days
Adam learning rate	0.001
Epochs	100

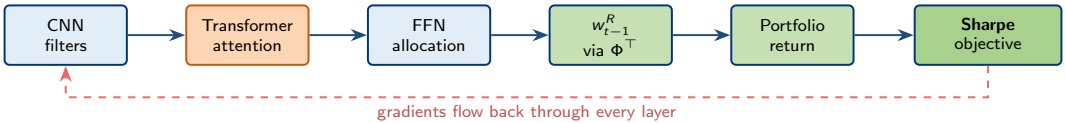
Parameter count (our count)

CNN layer 1	24
CNN layer 2	136
Attention projections	216
Head mixing W^O	72
Time-wise FFN	280
Layer norms	32
Final linear layer	9
Total	**769**

Sixteen candidate settings gave statistically indistinguishable validation Sharpes (3.5–4.2), so the paper chose the **most parsimonious** among them.

▶ **In finance:** low signal-to-noise calls for small models. Under 800 parameters is several orders of magnitude below a language Transformer.

Training: Sharpe as the loss function



Objective over a training window

$$\text{Sharpe} = \frac{\mathbb{E}\big[(w^R_{t-1})^\top R_t\big]}{\sqrt{\text{Var}\big((w^R_{t-1})^\top R_t\big)}}$$

Or mean–variance: $\mathbb{E}[\cdot] - \gamma \text{Var}(\cdot)$, optionally net of trading costs. Sample moments replace the expectations.

Procedure

- **Rolling**: train on 1000 days, re-estimate every 125.
- **Batches**: consecutive 125-day blocks of all stocks.
- **No prediction target**: it learns "history → weights" directly.
- About 7 hours for the full run on 8 GPUs.

▶ **In finance:** the model is judged on the metric the desk cares about, with no intermediate return forecast to calibrate.

Roadmap

The Big Picture

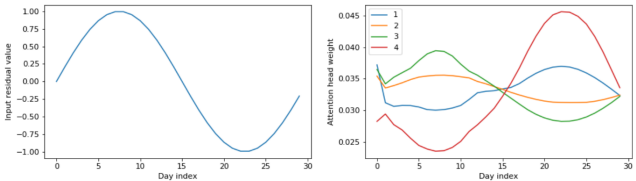
Portfolios, Signals, and Allocation

The CNN + Transformer Architecture

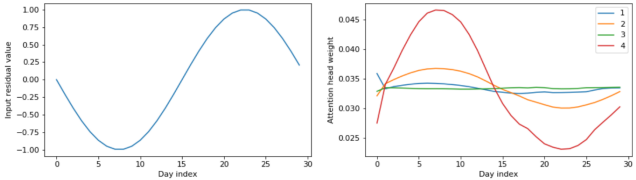
What the Attention Learns

Empirical Results

Attention discovers cycles it was never shown



(a) Input residual and attention head weights for $x_l = \sin\left(2\pi \frac{l}{30}\right)$



(b) Input residual and attention head weights for for $x_l = \sin\left(2\pi \frac{l+15}{30}\right)$

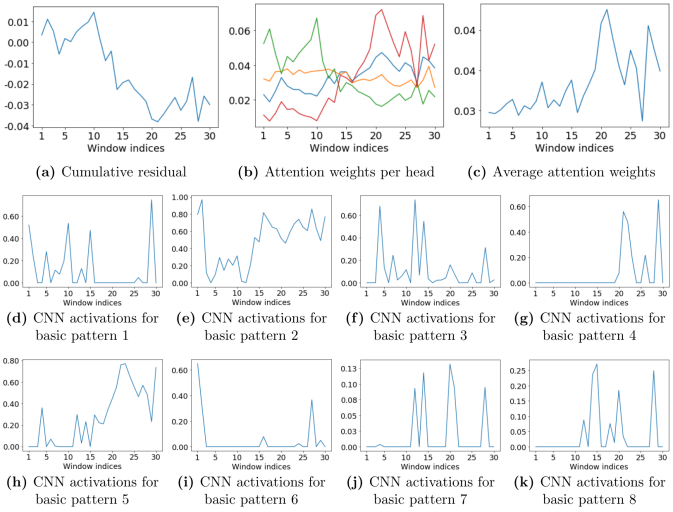
Experiment: feed in simulated sine waves to a model trained on real residuals.

- **Head 4**: strongest, high on negative realizations. "Negative reversal".
- **Head 3**: fires on a hilltop only early in the window. "Early reversal".
- **Head 1**: a dampened version of head 4.

Different heads capture different global patterns, as hoped.

Source: paper, Figure 15. Both sines have a 30-day cycle; the second is shifted by 15 days.

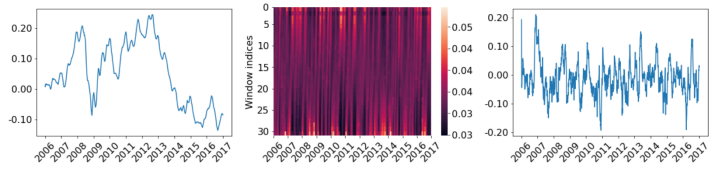
Dissecting one residual: from local patterns to attention



- **(d)–(k)**: CNN output, one activation series per basic pattern. This is the input to the Transformer.
- **(b)**: head 4 peaks at the bottoms; head 3 spikes at the early top.
- **(c)**: on average, attention leans to the latter half of the window.

Source: paper, Figure 16. One randomly selected, representative 30-day IPCA 5-factor residual.

Escalators up, elevators down



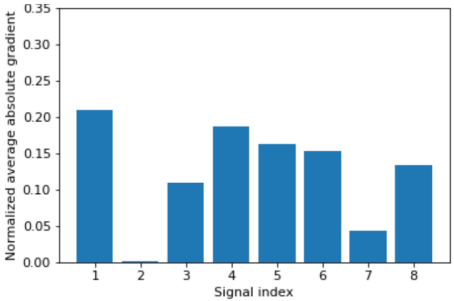
(a) Cumulative residuals (b) Average attention weights (c) Allocation weights

(d) Attention weights for head 1 (e) Attention weights for head 2 (f) Attention weights for head 3 (g) Attention weights for head 4

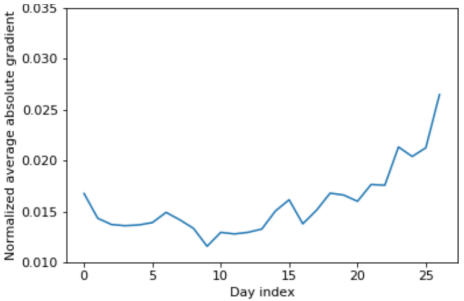
- In **uptrends**, attention focuses further back in the window. In **downtrends**, it focuses on the most recent days.
- The model learned to **act swiftly in downtrends and more slowly in uptrends**.
- This asymmetry is hard to replicate with a parametric OU model or fixed basis patterns.

Source: paper, Figure 17. A single residual, 2006–2016, smoothed with a 30-day moving average.

Which patterns and days matter?



(a) Importance of Local Basic Patterns



(b) Importance of Residual Days

- **(a)** The flat basic pattern 2 has negligible weight. Trend and reversal patterns carry the importance.
- **(b)** All earlier days matter, and the most recent 14 receive more attention on average.

Source: paper, Figure 18. Normalized average absolute gradient of the allocation weight.

Roadmap

The Big Picture

Portfolios, Signals, and Allocation

The CNN + Transformer Architecture

What the Attention Learns

Empirical Results

Data setup

Universe and sample

- **CRSP** daily US returns, Jan 1978–Dec 2016
- Pre-1998: used to estimate the factor models
- Jan 1998–Dec 2016: **daily residuals**
- Jan 2002–Dec 2016: **out-of-sample trading**
- Risk-free rate: 1-month T-bill

Liquidity filter

- Prior-month market cap $> 0.01\%$ of the total
- About the **550 largest stocks**, roughly the S&P 500
- Unbalanced: membership can change monthly

Firm characteristics

- **46 characteristics** from Chen et al. (2022), built from CRSP/Compustat and past returns
- Cross-sectionally **centered and rank-transformed** each month, which is robust to outliers and scale
- Lagged values feed the IPCA factor model

Daily series for trading

- **Adjusted returns**: dividends and splits included
- Trades at the close of each day
- Rolling factor regressions use only stocks with no missing returns in the window (removes at most 2%)

The 46 firm characteristics

	Past Returns	
(1)	r2_1	Short-term momentum
(2)	r12_2	Momentum
(3)	r12_7	Intermediate momentum
(4)	r36_13	Long-term momentum
(5)	ST_Rev	Short-term reversal
(6)	LT_Rev	Long-term reversal
	Investment	
(7)	Investment	Investment
(8)	NOA	Net operating assets
(9)	DPI2A	Change in property, plants, and equipment
(10)	NI	Net Share Issues
	Profitability	
(11)	PROF	Profitability
(12)	ATO	Net sales over lagged net operating assets
(13)	CTO	Capital turnover
(14)	FC2Y	Fixed costs to sales
(15)	OP	Operating profitability
(16)	PM	Profit margin
(17)	RNA	Return on net operating assets
(18)	ROA	Return on assets
(19)	ROE	Return on equity
(20)	SGA2S	Selling, general and administrative expenses to sales
(21)	D2A	Capital intensity
	Intangibles	
(22)	AC	Accrual
(23)	OA	Operating accruals
(24)	OL	Operating leverage
(25)	PCM	Price to cost margin
	Value	
(26)	A2ME	Assets to market cap
(27)	BEME	Book to Market Ratio
(28)	C	Ratio of cash and short-term investments to total assets
(29)	CF	Free Cash Flow to Book Value
(30)	CF2P	Cashflow to price
(31)	D2P	Dividend Yield
(32)	E2P	Earnings to price
(33)	Q	Tobin's Q
(34)	S2P	Sales to price
(35)	Lev	Leverage
	Trading Frictions	
(36)	AT	Total Assets
(37)	Beta	CAPM Beta
(38)	IdioVol	Idiosyncratic volatility
(39)	LME	Size
(40)	LTurnover	Turnover
(41)	MktBeta	Market Beta
(42)	Rel2High	Closeness to past year high
(43)	Resid_Var	Residual Variance
(44)	Spread	Bid-ask spread
(45)	SUV	Standard unexplained volume
(46)	Variance	Variance

Source: paper, Table A.I. Construction details are in the Internet Appendix of Chen, Pelger, Zhu (2022).

Main results (1)

Residuals are essential

- Trading raw returns ($K = 0$) is much worse than trading factor-model residuals.
- Beyond $K \approx 5$ factors, adding more brings little: a small number of factors capture most commonality.

CNN + Transformer dominates the benchmarks

- With IPCA residuals and $K = 5$, it reaches an **out-of-sample Sharpe of about 4**.
- That is roughly $2\times$ Fourier + FFN and $4\times$ OU + Threshold.

High return, not just low volatility

- Stock weights satisfy the strict **L1 leverage** limit of 1.
- Under that limit the model still delivers high annual means with moderate volatility, e.g. about 14% on PCA residuals.

Robust to the factor model

- It performs well with Fama–French, PCA, and IPCA residuals.
- Fama–French and PCA give similar Sharpes, suggesting they capture similar co-movement.
- **IPCA gives the highest Sharpes**: conditional factors capture more information.

Out-of-sample performance

		Fama–French			PCA			IPCA		
Model	K	SR	μ	σ	SR	μ	σ	SR	μ	σ
CNN + Trans	0	1.64	13.7%	8.4%	1.64	13.7%	8.4%	1.64	13.7%	8.4%
	1	3.68	7.2%	2.0%	2.74	15.2%	5.5%	3.22	8.7%	2.7%
	3	3.13	5.5%	1.8%	3.56	16.0%	4.5%	3.93	8.6%	2.2%
	5	3.21	4.6%	1.4%	3.36	14.3%	4.2%	**4.16**	8.7%	2.1%
	8	2.49	3.4%	1.4%	3.02	12.2%	4.0%	3.95	8.2%	2.1%
	10	–	–	–	2.81	10.7%	3.8%	3.97	8.0%	2.0%
	15	–	–	–	2.30	7.6%	3.3%	4.17	8.4%	2.0%
Fourier + FFN	5	1.66	3.1%	1.8%	1.98	12.4%	6.3%	1.90	7.7%	4.1%
OU + Thresh	5	0.38	0.9%	2.3%	0.73	4.4%	6.1%	0.97	3.8%	4.0%

Annualized Sharpe (SR), mean (μ), and volatility (σ), Jan 2002–Dec 2016. Sharpe objective, $L = 30$, no trading costs. $K = 0$ trades stock returns directly. Benchmarks are shown at $K = 5$ only.

Source: paper, Table I.

Main results (2)

Not a disguised risk-premium strategy

- Regressing returns on the **8-factor Fama–French** model gives large, highly significant alphas.
- Very low R^2 on FF and PCA residual strategies: essentially orthogonal to standard risk factors.
- Even with higher correlation in the IPCA case, **pricing errors remain significant**.

Stable through crises

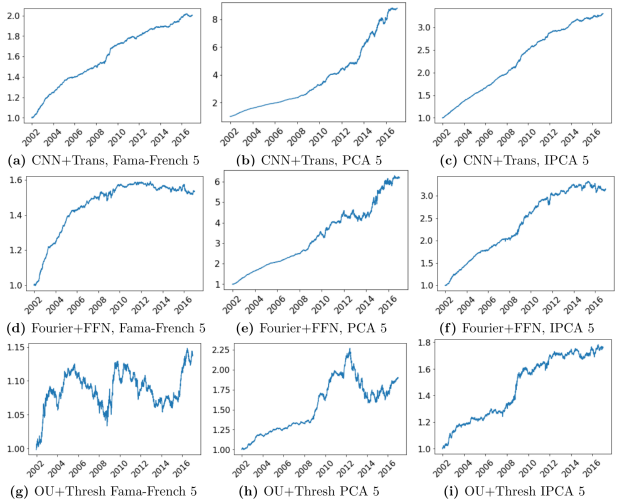
- Consistent, mostly positive out-of-sample cumulative returns, 2002–2016.
- Largely unaffected by the 2007 **quant quake** and the weak 2011–2012 quant period.
- Fourier + FFN has higher risk after the financial crisis; OU is visibly inferior.

Pricing errors: CNN + Trans

	FF		PCA		IPCA	
K	α	R^2	α	R^2	α	R^2
0	11.6%	30.3%	11.6%	30.3%	11.6%	30.3%
1	7.0%	2.4%	14.9%	0.6%	8.1%	9.5%
3	5.5%	1.2%	15.8%	1.7%	8.2%	6.0%
5	4.5%	2.3%	14.1%	1.3%	8.3%	3.9%
8	3.3%	2.1%	12.0%	0.9%	7.8%	5.0%
10	–	–	10.5%	0.7%	7.7%	4.0%
15	–	–	7.5%	0.5%	8.1%	4.2%

All α significant at 0.1% (t_α from 6.4 to 16). Source: paper, Table II.

Cumulative out-of-sample returns



(a) CNN+Trans, Fama-French 5 (b) CNN+Trans, PCA 5 (c) CNN+Trans, IPCA 5

(d) Fourier+FFN, Fama-French 5 (e) Fourier+FFN, PCA 5 (f) Fourier+FFN, IPCA 5

(g) OU+Thresh Fama-French 5 (h) OU+Thresh PCA 5 (i) OU+Thresh IPCA 5

Source: paper, Figure 5.

- **Top row**: CNN + Transformer, smooth and almost always rising.
- **Middle**: Fourier + FFN, similar until the financial crisis, riskier after.
- **Bottom**: OU + Threshold, visibly inferior.

All three trade the **same 5-factor residuals**, so the differences come from the policy.

Reality check: trading frictions

Costs subtracted from returns, in training and evaluation

$$\text{cost} = 0.0005 \, \| w^R_{t-1} - w^R_{t-2} \|_1 + 0.0001 \, \| \min(w^R_{t-1}, 0) \|_1$$

- 5 bps per transaction, 1 bp per short position
- No market impact, as the universe is liquid large caps

CNN + Trans, IPCA residuals, Sharpe objective

K	0	1	3	5	10	15
SR, no costs	1.64	3.22	3.93	4.16	3.97	4.17
SR, with costs	0.52	0.85	1.24	1.11	0.98	0.94

Source: paper, Tables I and IX.

Reading the table

- Sharpes fall by about two thirds or more. The headline number of 4 is **before costs**.
- At $K = 5$, the mean falls from 8.7% to 5.5% and the volatility rises from 2.1% to 5.0%.
- The strategy **remains profitable**, with Sharpes above 1 at $K = 3$ and $K = 5$.
- Factor residuals still beat raw returns ($K = 0$).

The network adapts: with costs in the objective, it lowers turnover and short exposure overall, but still trades more in turbulent periods such as 2007–2009.

Core message

For statistical arbitrage

- Profitable arbitrage trading requires both:
  1. a good **factor model** to define the residual portfolios;
  2. a powerful, data-driven **signal + allocation policy**.
- Among all approaches tested, **CNN + Transformer + FFN** is the most effective at extracting and trading short-horizon structure from factor-neutral residuals.

For Transformers

- The block from Part I works on **a noisy financial series** when kept **very small**: 1 layer, 8 dimensions, under 800 parameters.
- A **CNN embedding** gives it local structure before attention finds the global structure.
- Attention weights provide a window into the model: **an asymmetric policy emerges from the data**.
- Replacing the loss with **Sharpe** turns a forecasting tool into a trading policy.

Replication repo

github.com/omroot/DeepLearningStatisticalArbitrage_PaperReplication

The core of `src/models/cnn_transformer.py`, with benchmark values filled in:

```
encoder_layer = nn.TransformerEncoderLayer(
    d_model=8, nhead=4, dim_feedforward=16, dropout=0.25)
self.transformer = nn.TransformerEncoder(encoder_layer, num_layers=1)
self.output_proj = nn.Linear(8, 1)

x = self.cnn(x.unsqueeze(1))          # (N, 1, T) -> (N, 8, T)
x = x.permute(2, 0, 1)                # (T, N, 8): attention runs over T
x = self.transformer(x)
weights = self.output_proj(x[-1])     # last day only -> one weight per asset
```

Every idea of this lecture fits in those lines: tokens are days, N is a batch dimension, and only the last row is read.

References

- Jorge Guijarro-Ordonez, Markus Pelger, Greg Zanotti (2022), *Deep Learning Statistical Arbitrage*, arXiv:2106.04028
- Ashish Vaswani et al. (2017), *Attention Is All You Need*, NIPS
- Marco Avellaneda, Jeong-Hyun Lee (2010), *Statistical Arbitrage in the US Equities Market*, Quantitative Finance
- J. Yeo, G. Papanicolaou (2017), *Risk Control of Mean-Reversion Time in Statistical Arbitrage*, Risk and Decision Analysis
- Bryan Kelly, Seth Pruitt, Yinan Su (2019), *Characteristics Are Covariances: A Unified Model of Risk and Return*, Journal of Financial Economics
- Luyang Chen, Markus Pelger, Jason Zhu (2022), *Deep Learning in Asset Pricing*, Management Science